

UNIVERSIDADE FEDERAL FLUMINENSE
INSTITUTO DE COMPUTAÇÃO
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

ATHARV NUTHI
FELIPE VAHIA MALLIAGROS

A Evolução do Ecossistema de Software via Inteligência Artificial: Análise Multidimensional e a Proposição da Topologia PASE

Niterói 2026



A Evolução do Ecossistema de Software via Inteligência Artificial: Análise Multidimensional e a Proposição da Topologia PASE

Trabalho de conclusão de curso de graduação apresentado ao Departamento de Ciência da Computação da Universidade Federal Fluminense como requisito parcial para a obtenção do título de Bacharel(a) em Ciência da Computação.

Orientador(a): Prof. Troy Costa Kohwalter

Niterói 2026

FICHA CATALOGRÁFICA

Inserir aqui a ficha gerada a partir do Sistema de Geração Automática de Fichas Catalográficas, disponível no endereço https://bibliotecas.uff.br/bee/fichacatalografica

Niterói 2026

ATHARV NUTHI

FELIPE VAHIA MALLIAGROS

A Evolução do Ecossistema de Software via Inteligência Artificial: Análise Multidimensional e a Proposição da Topologia PASE

Trabalho de conclusão de curso de graduação apresentado ao Departamento de Ciência da Computação da Universidade Federal Fluminense como requisito parcial para a obtenção do título de Bacharel(a) em Ciência da Computação.

Aprovado em: ____ de _____ de 2026.

BANCA EXAMINADORA

_____

Nome do professor - instituição (orientador)

_____

Nome do professor - instituição

_____

Nome do professor - instituição

Niterói 2026

AGRADECIMENTOS

RESUMO

O presente trabalho tem como objetivo geral analisar de forma multidimensional o impacto da Inteligência Artificial no ecossistema de software, investigando a transição para modelos baseados em agentes, a democratização do desenvolvimento, a evolução da experiência do usuário, os riscos de segurança/governança e as transformações no mercado de trabalho.

As questões centrais de pesquisa que norteiam este estudo são: “A Inteligência Artificial transformará o software em um ecossistema puramente autônomo de agentes, substituindo progressivamente os modelos tradicionais, ou criará híbridos humano-IA que demandam novas topologias, competências e governança?” e como uma nova topologia pode ser aplicada de forma prática para gerenciar essa transição.

A metodologia empregada consiste em uma revisão sistemática da literatura publicada entre 2017 e 2026, seguida por uma pesquisa propositiva para a construção de um novo framework.

Como contribuição central, o estudo propõe a topologia PASE (Personalized Adaptive Software Ecosystem), um framework de três camadas (Task, Orchestration, Execution) que equilibra a autonomia sistêmica com a supervisão humana necessária, garantindo que o "veto humano" seja um componente integral e obrigatório na camada de execução. A PASE oferece uma solução prática para implementar sistemas de IA agênticos de forma segura, governável e alinhada às necessidades humanas, servindo como um roteiro para a transição para o Software Engineering 2.0.

Palavras-chave: Inteligência Artificial Agêntica, Engenharia de Software, Ciclo de Vida de Desenvolvimento de Software (SDLC), Sistemas Multi-Agente, Topologia PASE, Governança de IA

ABSTRACT

This project aims to provide a multidimensional analysis of the impact of Artificial Intelligence on the software ecosystem, investigating the transition to agent-based models, the democratization of development, the evolution of user experience, security/governance risks, and transformations in the job market.

The central research questions guiding this study are: "Will Artificial Intelligence transform software into a purely autonomous ecosystem of agents, progressively replacing traditional models, or will it create human-IA hybrids that demand new topologies, skills, and governance?" and how can a new topology be practically applied to manage this transition.

The methodology employed consists of a systematic literature review published between 2017 and 2026, followed by a propositional research for the construction of a new framework.

As a central contribution, the study proposes the PASE (Personalized Adaptive Software Ecosystem) topology, a three-layer framework (Task, Orchestration, Execution) that balances systemic autonomy with necessary human supervision, ensuring that the "human veto" is an integral and mandatory component in the execution layer. PASE offers a practical solution for implementing agentic AI systems in a secure, governable, and human-aligned manner, serving as a roadmap for the transition to Software Engineering 2.0.

Keywords: Agentic Artificial Intelligence, Software Engineering, Software Development Life Cycle (SDLC), Multi-Agent Systems, PASE Topology, AI Governance

LISTA DE ILUSTRAÇÕES

Figura 1 – Criação de valor acumulado projetada impulsionada por IA..... 09
Figura 2 – Título..... 26
Figura 3 – Título..... 33
Figura 4 – Título..... 45
Figura 5 – Título..... 56

LISTA DE TABELAS

Tabela 1 – Principais lacunas por dimensão na transição para sistemas de IA agêntica.. 20

SUMÁRIO

1. INTRODUÇÃO **6**
2. SOFTWARE NA ERA DA IA AGÊNTICA: IMPACTOS E LACUNAS **8**
2.1 Indústria e Aplicações 8
2.2 Desenvolvimento de Software 11
2.3 Usuário e Experiência 13
2.4 Segurança, Ética e Governança 15
2.5 Mercado de Trabalho 17
2.6 Síntese 20
3. TOPOLOGIA PASE: PERSONALIZED ADAPTIVE SOFTWARE ECOSYSTEM **22**
3.1 Introdução e Visão Geral da Arquitetura 22
3.2 Camada de Tarefas (Task Layer) 24
3.2.1 Formação de Requisitos 24
3.2.2 Avaliador de Zonas Dinâmicas 26
3.2.3 Motor de Filtragem de tarefas 27
3.3 Camada de Orquestração (Orchestration Layer) 30
3.3.1 Motor de Orquestração 31
3.4 Camada de Execução (Execution Layer) 34
3.4.1 Executor de Tarefas 34
3.4.2 User Personalization Agent (UPA) 37
4. CONCLUSÃO **40**
REFERÊNCIAS **43**
APÊNDICE A **47**
ANEXO A **47**

1. INTRODUÇÃO

O ecossistema de software, que sustenta grande parte da economia digital atual, vive uma transformação profunda impulsionada pela Inteligência Artificial. Modelos de linguagem de grande escala e, mais recentemente, sistemas agênticos estão mudando não só a forma como o software é criado, mas também sua natureza essencial. O que antes era uma ferramenta passiva, usada por humanos para executar tarefas, torna-se cada vez mais um ator autônomo ou semi-autônomo capaz de planejar, decidir e agir. Essa passagem de um paradigma internet-based para um modelo agent-based marca uma das mudanças mais significativas na história da computação. O software deixa de ser apenas suporte e passa a executar trabalho de forma independente (SAROYA; MACNEILL, 2026). O resultado promete ganhos de produtividade expressivos, porém traz consigo desafios complexos de controle, segurança e alinhamento com objetivos humanos.

A engenharia de software, historicamente centrada na escrita manual e na depuração, evolui agora para uma prática de orquestração colaborativa entre humanos e agentes de IA (ANTHROPIC, 2026). Ferramentas assistidas por IA já participam de forma relevante na produção de código novo, democratizando o acesso ao desenvolvimento (GOOGLE CLOUD, 2025). Mesmo assim, essa automação carrega limitações claras. Estudos demonstram que modelos de linguagem frequentemente desrespeitam princípios básicos de modularidade e encapsulamento, gerando soluções que parecem funcionais mas escondem dependências e aumentam a complexidade desnecessária (KRAVCHUK-KIRILYUK et al., 2025). Integrar IA em todas as etapas do ciclo de vida de desenvolvimento exige, portanto, cautela, mecanismos rigorosos de validação e, acima de tudo, supervisão humana em decisões críticas (BURTE, 2025).

Diante desse cenário, surge uma questão central: a Inteligência Artificial caminhará para um ecossistema totalmente autônomo de agentes, substituindo gradualmente os modelos tradicionais, ou dará origem a sistemas híbridos humano-IA que exigem novas arquiteturas, competências e formas de governança? A resposta envolve múltiplas dimensões e afeta diretamente a indústria, a prática de desenvolvimento, a experiência dos usuários, a segurança e o mercado de trabalho. Sem uma gestão adequada, a proliferação de agentes pode gerar um cenário de fragmentação descontrolada, no qual a falta de coordenação compromete a confiança e o retorno sobre o investimento (SPARK, 2026).

Para lidar com essa complexidade, coloca-se uma segunda questão, mais prática: como projetar uma topologia de software capaz de equilibrar a autonomia crescente dos sistemas de IA com a necessidade permanente de supervisão, controle e alinhamento humano? A literatura e a prática industrial ainda carecem de frameworks que estruturem essa interação de forma consistente. A eficácia de sistemas multi-agente depende menos do número de agentes e mais do alinhamento entre a arquitetura de coordenação e a natureza da tarefa (KIM et al., 2026). Da mesma forma, a confiança dos usuários diminui quando a personalização ocorre sem transparência (JAMALI et al., 2025).

Este trabalho surge exatamente dessa dualidade. Em 2026, o uso de IA no desenvolvimento de software é amplamente difundido, e a transição para sistemas autônomos já se consolida em boa parte das organizações (GOOGLE CLOUD, 2025; SPARK, 2026). Ao mesmo tempo, ameaças cibernéticas impulsionadas por IA se intensificam, reforçando a importância da governança. Este estudo examina tanto o potencial transformador quanto os riscos da era agêntica, por meio de uma revisão sistemática da literatura e de relatórios recentes entre 2017 e 2026, combinada com a análise de cinco dimensões principais: indústria e aplicações, desenvolvimento de software, experiência do usuário, segurança ética e governança, e mercado de trabalho. A partir dessa base, propõe-se uma topologia híbrida, denominada PASE, como contribuição original para preencher a lacuna identificada e orientar a construção de sistemas que sejam ao mesmo tempo capazes e confiáveis.

2. SOFTWARE NA ERA DA IA AGÊNTICA: IMPACTOS E LACUNAS

2.1 Indústria e Aplicações

A história do software empresarial é uma sucessão de paradigmas que, a cada ciclo, não só mudou a arquitetura dos sistemas, mas redefiniu o tipo de trabalho que eles realizam. Do software de gestão instalado localmente nos anos 1990, passando pela computação em nuvem e pelo modelo SaaS que marcou as décadas de 2000 e 2010, cada transição ampliou o mercado e alterou a cadeia de valor das organizações. A mudança em curso (do software internet-based para o AI-based e, cada vez mais, para o modelo agent-based) representa, porém, uma ruptura de natureza diferente.

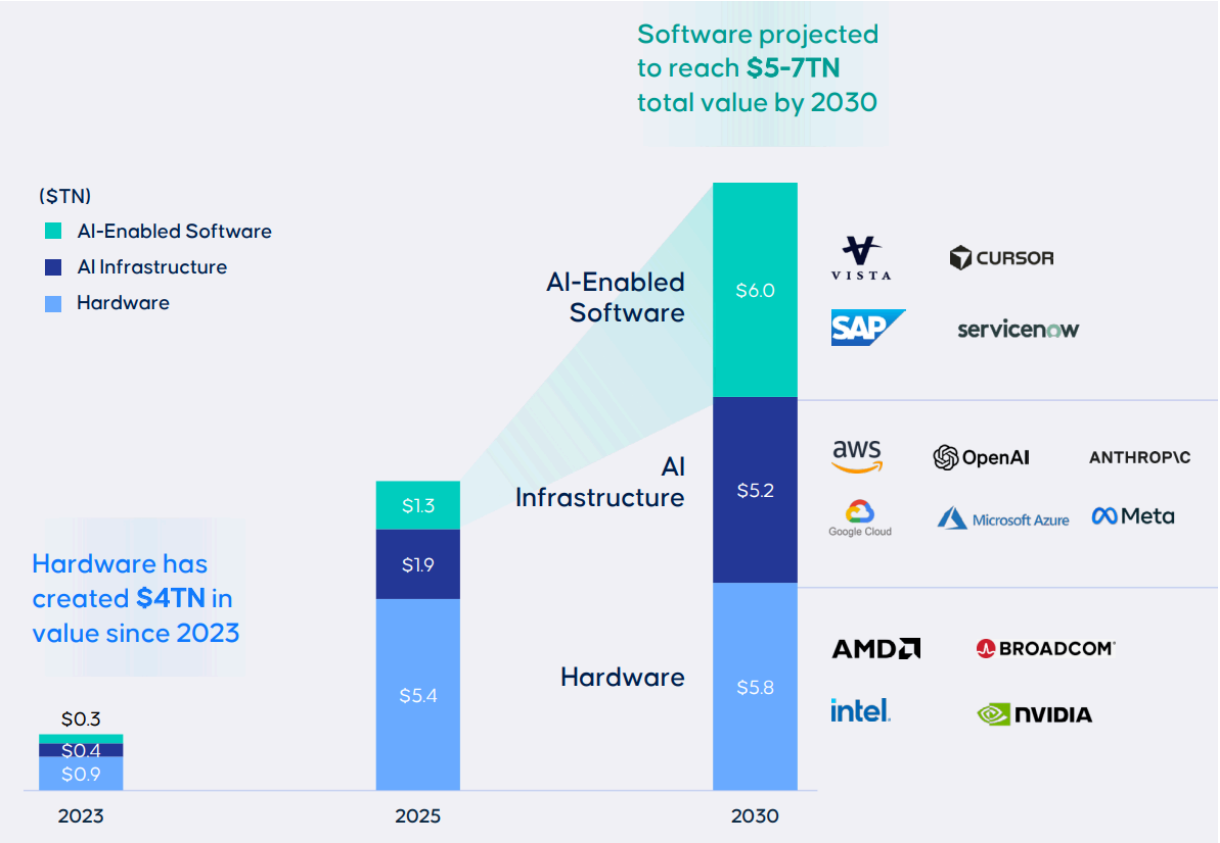
Pela primeira vez, o software deixa de ser apenas uma ferramenta que apoia o trabalho humano para se tornar um agente que executa trabalho de forma autônoma (SAROYA; MACNEILL, 2026). Essa diferença traz uma reconfiguração profunda nos processos de negócio, nos modelos de precificação e nas competências que as empresas precisam ter para competir ou, em alguns casos, até para sobreviver.

No paradigma internet-based que predominou até meados da década passada, o software funcionava basicamente como interface e repositório: capturava dados, armazenava e apresentava para o usuário humano, que tomava as decisões. Sistemas de CRM, ERP e plataformas de colaboração são exemplos clássicos. O modelo AI-based, que ganhou força a partir de 2017, acrescentou uma camada de inteligência analítica, permitindo recomendações, predições e automações pontuais. Já o modelo agent-based, que começa a se consolidar industrialmente entre 2024 e 2025, dá um salto qualitativo: agentes autônomos, construídos com modelos de linguagem de grande escala (LLMs), memória persistente, ferramentas de integração com sistemas e capacidade de planejamento multi-etapa, conseguem decompor objetivos complexos, coordenar sub-agentes especializados e executar ações em ambientes de produção sem necessidade de intervenção humana constante (ANTHROPIC, 2026).

A escala dessa transição aparece nos números. Projeções da Bain & Company estimam que entre US$5 e 7 trilhões em valor acumulado devem fluir para a camada de software e aplicações até 2030, mais que o quádruplo dos níveis atuais e, potencialmente, a

maior expansão de mercado endereçável da história do software, conforme ilustrado na Figura 1. No lado do investimento corporativo, a contribuição total em IA triplicou em apenas um ano, chegando a US$37 bilhões em 2025, com a camada de aplicação ficando com mais da metade de todos os recursos (SAROYA; MACNEILL, 2026).

Figura 1 - Criação de valor acumulado projetada impulsionada por IA



Fonte: Bain & Company (2024).

Levantamentos de larga escala confirmam a velocidade da adoção: a pesquisa do Google Cloud com quase cinco mil profissionais de tecnologia mostrou que 90% dos respondentes já utilizavam IA no trabalho (GOOGLE CLOUD, 2025). Dados do McKinsey Global Institute vão na mesma direção: 88% das organizações relataram uso regular de IA em pelo menos uma função de negócio, e 62% já experimentam ou utilizam agentes de IA em algum grau (MCKINSEY, 2025b). O Gartner projeta que 40% dos aplicativos empresariais contarão com agentes de IA para tarefas específicas até o final de 2026 (GARTNER, 2025a; GARTNER, 2026).

Em diversos setores industriais, essa transição já está produzindo resultados mensuráveis. Na defesa e na manufatura de alta complexidade, a Palantir implantou sua plataforma AIP com arquitetura de agentes em camadas, permitindo que sistemas autônomos agreguem dados de ERP, bancos legados e fontes operacionais para identificar gargalos em cadeias de suprimentos e otimizar cronogramas de projetos de grande porte. No setor de energia nuclear, parcerias semelhantes integraram agentes para otimizar controles de projetos e manufatura em instalações classificadas, demonstrando que sistemas agentic podem atuar de forma confiável em contextos de alta regulamentação e risco. Na indústria aeroespacial, a GE Aerospace expandiu o uso de agentes ao longo de toda a cadeia de fornecimento, alcançando ganhos expressivos de produtividade no desenvolvimento de software e melhoria significativa nos prazos de entrega de sistemas de monitoramento de motores.

Tendências paralelas aparecem em serviços financeiros e operações de grande escala. A Duck Creek Technologies desenvolveu “Uma”, uma arquitetura de agentes especializados que automatiza subscrição, análise de riscos e conformidade regulatória dentro de fluxos estruturados e auditáveis, processando volumes anuais da ordem de dezenas de bilhões de dólares em prêmios (SAROYA; MACNEILL, 2026). No recrutamento, plataformas como a Fountain implantaram agentes que aceleraram triagem de candidatos e processos de onboarding, enquanto operadoras de telecomunicações como a TELUS registraram economia de centenas de milhares de horas de trabalho e aceleração na produção de código por meio de sistemas agênticos. Em automação de workflows, a Zapier relatou adoção interna massiva de agentes que executam processos de ponta a ponta em vendas, marketing e suporte jurídico sem intervenção contínua de equipes técnicas. Um caso particularmente ilustrativo veio da Rakuten, onde um agente operou de forma autônoma por horas contínuas sobre uma base de código de milhões de linhas em múltiplas linguagens, alcançando alta precisão numérica, evidenciando que agentes de longa duração já conseguem lidar com complexidade organizacional real (ANTHROPIC, 2026).

Esses casos evidenciam que a IA agêntica não opera como um conjunto de ferramentas pontuais, mas como infraestrutura de orquestração que decompõe processos organizacionais complexos em grafos de tarefas direcionados, distribui a execução entre sub-agentes especializados e coordena a síntese dos resultados. A transição de agentes únicos

para sistemas multi-agente com especialização, paralelismo e protocolos de coordenação representa a fronteira técnica atual da IA industrial (ANTHROPIC, 2026).

Apesar desses avanços, a transição para ecossistemas agent-based ainda expõe lacunas importantes. O “paradoxo da IA generativa” evidencia que, embora quase oito em cada dez empresas utilizem IA generativa, o mesmo percentual relata ausência de impacto mensurável no resultado operacional, com aproximadamente 90% dos casos de uso verticais transformadores permanecendo presos em fases de experimentação e piloto (MCKINSEY, 2025a). A pesquisa DORA do Google Cloud (2025) mostra que a IA funciona como amplificador: ela magnifica tanto as forças quanto as instabilidades das organizações, podendo anular ganhos de throughput quando não há capacidades maduras de governança (GOOGLE CLOUD, 2025). O Gartner alerta que mais de 40% dos projetos de IA agêntica podem ser cancelados até o final de 2027 por escalada de custos, valor indefinido ou controles de risco insuficientes (GARTNER, 2025b). Essa dualidade (capacidade técnica já demonstrada versus ausência de frameworks estruturados de integração e supervisão) aumenta o risco de Agent Sprawl: proliferação descontrolada de agentes que fragmenta arquiteturas, erode confiança e compromete o retorno sobre o investimento.

Por fim, a análise da dimensão industrial revela uma convergência inédita de capacidade técnica, escala de investimento e pressão competitiva que acelera a transição do ecossistema de software para o paradigma agent-based. No entanto, essa adoção massiva ainda ocorre majoritariamente sem uma topologia arquitetural unificada para integração de agentes autônomos, preservação do controle humano em processos críticos e estruturação adequada de governança e rastreabilidade. A literatura reforça a necessidade de capacidades organizacionais fundacionais (como ecossistemas de dados saudáveis, controle de versão robusto e foco centrado no usuário) para que os ganhos potenciais se materializem (GOOGLE CLOUD, 2025; BURTE, 2025). Essa lacuna específica na dimensão industrial evidencia a importância de frameworks estruturados para orientar a implementação responsável, tema que será aprofundado nas dimensões seguintes.

2.2 Desenvolvimento de Software

A engenharia de software sempre viveu uma tensão entre a promessa de automação e a complexidade intrinsecamente humana do processo criativo. Desde o modelo em cascata até as metodologias ágeis, cada paradigma buscou organizar e racionalizar essa complexidade. Em todos eles, no entanto, o ser humano permanecia como o principal agente de decisão, criatividade e responsabilidade sobre o resultado final.

A adoção generalizada de LLMs e sistemas agênticos representa uma ruptura nesse padrão. A IA não se limita a apoiar o desenvolvimento. Ela passa a capturar e orquestrar fases inteiras do ciclo de vida de software (SDLC), desde a elicitação e estruturação de requisitos até a manutenção e evolução do sistema. Essa mudança altera quem decide, quem cria e quem responde pelos artefatos produzidos.

Os dados quantitativos revelam uma transformação rápida e de larga escala. Um relatório da Stack Overflow de 2024 documenta que 92% dos desenvolvedores globais utilizam ferramentas de IA na codificação diária, um crescimento de 28 pontos percentuais em relação a 2023. O GitHub reportou que os usuários da plataforma rejeitam aproximadamente 25-30% das sugestões de código geradas por IA, aceitando a maioria sem modificação substancial. A maioria dos desenvolvedores já utiliza essas ferramentas no cotidiano de trabalho, com redução expressiva no tempo de ciclo de revisão de código e ganhos de produtividade reportados entre 30% e 45% em tarefas específicas.

A geração de código é onde a transformação é mais evidente, e também mais controvertida. Uma revisão sistemática de 395 artigos acadêmicos sobre LLMs para engenharia de software (LLM4SE) publicada em 2025 mapeou 112 tarefas específicas cobertas pelas pesquisas, descobrindo que a geração de código representava 56,65% de toda a pesquisa acadêmica na área, seguida por manutenção de software em 22,71%. Esses números, embora impressionantes, escondem uma realidade complexa: os ganhos de produtividade no SDLC são altamente seletivos, avançando de forma muito mais acelerada na geração de código do que em fases estratégicas como engenharia de requisitos e arquitetura.

Esse desequilíbrio cria um risco concreto de degradação estrutural. A geração de código assistida por IA, apesar de ser a face mais visível da automação, frequentemente padece de Architectural Mimicry: os modelos produzem código que parece funcional na superfície, mas viola princípios fundamentais de modularidade e encapsulamento, mascarando uma perda gradual de qualidade arquitetural em sistemas de larga escala sob a aparência de eficiência.

Além da geração de código, a IA também está transformando a fase de testes, tanto na criação automática de casos quanto na priorização inteligente de regressões. Ferramentas especializadas já conseguem aumentar significativamente a cobertura de testes sem intervenção manual e reduzir drasticamente o tempo de execução de suítes de testes. No entanto, o impacto mais profundo da IA no desenvolvimento de software não está nas ferramentas em si, mas na redefinição do papel do engenheiro. A hipótese de substituição em massa de profissionais por máquinas não se confirma na prática. Estudos mostram que engenheiros que adotam essas ferramentas relatam, na maioria dos casos, um aumento, e não uma redução, de responsabilidades. O trabalho se desloca de escrever código linha a linha para atividades de maior nível de abstração: formulação precisa de prompts, orquestração de agentes, revisão crítica de qualidade e gestão ativa de dívida técnica.

A orquestração de sistemas multi-agente no contexto do SDLC representa tanto uma fronteira técnica promissora quanto um importante vetor de risco. Experimentos demonstram que agentes especializados podem ser coordenados para realizar tarefas complexas de forma autônoma. No entanto, a eficácia desses sistemas não é linear: ela depende fortemente da qualidade da decomposição do problema e da capacidade de coordenação entre os agentes. Quando essa coordenação é inadequada, o desempenho pode cair de forma significativa em relação a abordagens mais simples ou centralizadas. Essa variabilidade coloca a questão da governança e da supervisão humana no centro da discussão sobre o desenvolvimento assistido por IA.

Outros aspectos da transformação também merecem atenção. A democratização do desenvolvimento por meio de citizen developers traz ganhos reais de produtividade e inclusão, mas aumenta a exposição a riscos de qualidade e segurança quando não acompanhada de mecanismos de governança. Da mesma forma, vulnerabilidades de

segurança em código gerado por IA (incluindo falhas clássicas como SQL injection) aparecem com frequência relevante em análises, muitas vezes de forma não evidente para revisores humanos. O custo operacional da inferência em larga escala também começa a impactar o retorno real sobre o investimento em algumas organizações.

Esses movimentos, em conjunto, revelam que a transformação em curso é genuína, mas ainda incompleta. Os ganhos de velocidade e escala são reais e mensuráveis. Ao mesmo tempo, a qualidade arquitetural, a segurança e a supervisão humana em pontos críticos de decisão ainda carecem de frameworks mais robustos e estruturados. Essa concentração de avanços na geração de código, enquanto outras fases estratégicas do SDLC permanecem relativamente incipientes, reforça a segunda questão de pesquisa deste trabalho.

2.3 Usuário e Experiência

A história da interação humano-máquina pode ser lida como uma sucessão de libertações progressivas das restrições técnicas impostas pela máquina. Do cartão perfurado à interface gráfica e depois à web, cada etapa ampliou o acesso e reduziu a necessidade de conhecimento técnico especializado. A transição atual, no entanto, é de natureza diferente.

Com o avanço dos LLMs, surge a possibilidade de interfaces generativas e hiper-adaptativas: sistemas capazes de gerar, em tempo real, a própria interface, o conteúdo e os fluxos de interação com base na intenção expressa pelo usuário em linguagem natural. A interface deixa de ser um artefato fixo e pré-definido para se tornar uma superfície dinâmica que se reconfigura continuamente de acordo com o contexto, as preferências e os objetivos de cada pessoa (ARORA et al., 2025).

Essa capacidade técnica, porém, traz consigo transformações psicológicas e sociais que vão além da usabilidade. Estudos mostram que usuários conseguem realizar tarefas mais rapidamente em interfaces generativas, mas frequentemente relatam menor confiança no resultado obtido, mesmo quando a taxa de erro objetiva é equivalente à de interfaces tradicionais. Da mesma forma, há uma preferência declarada por esses sistemas, acompanhada de uma preocupação significativa com a falta de transparência, ou seja, a dificuldade de compreender por que uma determinada recomendação ou resultado foi gerado

(MUKTI; TRISILIA, 2025). Essa tensão entre velocidade e confiança, entre preferência e desconfiança, revela que a mudança não é apenas na interface, mas na própria relação entre o usuário e o sistema.

A personalização hiper-adaptativa aprofunda ainda mais essa transformação. Diferente de sistemas que apenas respondem a comandos explícitos, interfaces hiper-adaptativas antecipam necessidades, ajustam o nível de dificuldade, o ritmo e o estilo de apresentação com base em sinais comportamentais sutis. Experiências em contextos educacionais e corporativos demonstram ganhos claros de retenção e satisfação quando essa adaptação é bem executada (JAMALI et al., 2025). No entanto, essa mesma otimização expõe um trade-off importante entre eficiência e engajamento humano significativo.

Usuários relatam, em alguns casos, sensação de perda de autonomia e agência, a impressão de que o sistema está decidindo por eles. Em ambientes de trabalho, a automação inteligente de triagem e sumarização de informações pode reduzir o tempo de processamento, mas também diminui a exposição a conteúdos não otimizados, reduzindo a serendipidade e o senso de conexão com o trabalho dos colegas. Esse fenômeno se manifesta também na redefinição do papel de profissionais que historicamente atuavam como tradutores de informação complexa, como contadores, consultores e analistas. Sistemas de IA que realizam essa tradução de forma automatizada e personalizada tendem a comoditizar parte do valor que esses especialistas historicamente capturavam por meio de discernimento contextual e julgamento experiencial.

A confiança em sistemas autônomos depende de fatores como transparência, explicabilidade, histórico de acurácia, visibilidade de mecanismos de correção e presença percebida de supervisão humana. Quando esses elementos não estão presentes de forma clara, a disposição para delegar decisões diminui. Além disso, a exposição contínua a interfaces hiper-adaptativas parece alterar as expectativas dos usuários em relação a outros sistemas. Pessoas acostumadas com adaptação dinâmica tendem a avaliar interfaces estáticas como mais frustrantes e menos eficientes.

Outros aspectos dessa transformação merecem atenção. Embora sistemas generativos possam aumentar a acessibilidade ao oferecer múltiplas formas de representação de conteúdo,

eles também podem reproduzir vieses sociais, reforçando estigmas ou infantilizando pessoas com deficiência em descrições geradas automaticamente. Da mesma forma, a necessidade de capturar grandes volumes de dados comportamentais para alimentar a personalização cria tensões com a privacidade e o consentimento informado.

Por fim, observa-se o fenômeno de user experience reversion: usuários que experimentam interfaces hiper-adaptativas por um período e, posteriormente, solicitam voluntariamente o retorno a interfaces mais estáticas e previsíveis, alegando cansaço de serem constantemente monitorados ou a perda de espaço para descoberta e improviso (ARORA et al., 2025). Esse comportamento sugere que existe um limite para a otimização. Interfaces que se antecipam demais podem reduzir a agência humana e o sentido de controle, mesmo quando entregam resultados objetivamente superiores.

Essas dinâmicas revelam que a transição para interfaces generativas e hiper-adaptativas não é apenas uma evolução técnica. Ela traz consigo trade-offs entre eficiência e significado, entre personalização e agência, entre conveniência e privacidade. Quando a personalização ocorre sem transparência ou mecanismos claros de controle pelo usuário, a confiança é minada, um padrão que ecoa a tensão mais ampla entre autonomia dos sistemas e necessidade de supervisão humana.

2.4 Segurança, Ética e Governança

A segurança em software evoluiu historicamente por camadas sucessivas que se tornaram mais sofisticadas e, ao mesmo tempo, mais frágeis à medida que os sistemas ganharam complexidade e conectividade. No paradigma atual de IA agêntica, surge uma descontinuidade qualitativa. A defesa não se limita a proteger contra acesso não autorizado a dados ou sistemas. É preciso proteger também contra a manipulação de intenções: um adversário que não busca roubar informações, mas subverter o próprio processo de decisão autônoma do sistema.

A vetorização de ataque se expandiu. Em vez de injetar código malicioso em campos de entrada explícitos, um atacante pode enviar um prompt cuidadosamente elaborado que, ao ser processado por um LLM dentro de uma arquitetura multi-agente, altera comportamentos e

escala por toda a orquestração. A IA amplifica tanto a capacidade de defesa quanto a de ataque (CROWDSTRIKE, 2026).

De acordo com o CrowdStrike 2026 Global Threat Report, o mundo opera na era agentic. Em 2025, ataques por adversários habilitados por IA aumentaram 89% ano após ano. A IA acelerou phishing, reconnaissance e reduziu drasticamente o tempo entre acesso inicial e impacto. Adversários também passaram a mirar diretamente os próprios sistemas de IA que sustentam as empresas modernas, transformando-os em parte da superfície de ataque.

Vulnerabilidades específicas de sistemas baseados em LLM, como prompt injection, foram identificadas na grande maioria das implementações comerciais. Adversários manipulam sistemas habilitados por IA por meio de entradas que influenciam o comportamento do modelo de formas não intencionadas. Eles já experimentam prompt injection para interferir em workflows de segurança baseados em IA, inclusive embutindo prompts ocultos em e-mails de phishing para confundir triagem automatizada. A maioria dos agentes LLM é vulnerável a prompt injection, backdoors baseados em recuperação e explorações de confiança entre agentes (CHHABRA, 2026).

Outro vetor relevante é a alucinação e a geração insegura de código. Modelos avançados produzem código com taxas significativas de vulnerabilidades em funções críticas (autenticação, criptografia, validação de entrada), e frequentemente avaliam seu próprio código como seguro, revelando desconexão entre geração e avaliação de riscos.

No plano ético, a replicação e amplificação de vieses (algorithmic bias) representa um risco de longo prazo. Sistemas treinados em dados históricos tendem a reproduzir e intensificar padrões de discriminação, especialmente em contratação, crédito e serviços públicos. Os desafios éticos e de governança variam conforme o paradigma: sistemas simbólicos/clássicos predominam em domínios de segurança crítica (ex.: saúde), enquanto sistemas neurais/generativos prevalecem em ambientes adaptativos e ricos em dados (ex.: finanças). Isso revela lacunas significativas em modelos de governança para sistemas simbólicos e a necessidade urgente de arquiteturas híbridas neuro-simbólicas (ABOU ALI; DORNAIKA, 2025).

A governança de sistemas autônomos permanece um dos pontos mais frágeis. Governar um sistema que apenas recomenda é diferente de governar um sistema que executa ações. A rastreabilidade torna-se mais difícil devido às representações vetoriais de alta dimensão dos LLMs. A regulação avança de forma fragmentada (ex.: AI Act europeu), e a responsabilidade legal (liability) por danos causados por agentes autônomos ainda é ambígua. A privacidade também enfrenta desafios: sistemas hiper-adaptativos exigem captura contínua de dados comportamentais, tensionando princípios de minimização de dados e consentimento (GDPR).

Esses diferentes vetores (novos tipos de ataque, geração insegura de código, amplificação de vieses, fragilidade da governança, ambiguidade regulatória e tensão com a privacidade) mostram que a segurança, ética e governança de sistemas agênticos não podem ser tratadas apenas como problemas técnicos. Elas exigem uma abordagem sociotécnica, que integre tecnologia, processos organizacionais, supervisão humana em pontos críticos e estruturas de rastreabilidade robustas. Essa necessidade de supervisão humana em decisões de alto risco reforça a tensão central entre autonomia crescente dos sistemas e a indispensável manutenção de controle e alinhamento com objetivos humanos.

2.5 Mercado de Trabalho

Atualmente, a velocidade do deslocamento está superando, de forma clara, a capacidade histórica de reconversão da força de trabalho. Isso gera um hiato temporal em que milhões de profissionais podem ser deslocados simultaneamente sem que exista, no curto prazo, demanda suficiente por novas competências em suas regiões ou setores.

Os dados mais recentes confirmam essa aceleração. Estimativas globais apontam para dezenas de milhões de empregos em risco de deslocamento nos próximos anos, com a criação de um número ligeiramente superior de novas posições. No entanto, o saldo líquido positivo mascara um problema mais grave: os empregos que estão sendo eliminados e os que estão sendo criados não coincidem geograficamente, exigem competências distintas e ocorrem em momentos diferentes. Profissões de menor complexidade cognitiva, como suporte administrativo, atendimento e entrada de dados, sofrem deslocamento mais acentuado. Já

profissões de maior complexidade intelectual tendem a absorver parte da mudança em funções de maior valor agregado.

Essa desigualdade se manifesta também em escala global. Economias mais ricas tendem a apresentar correlação positiva entre automação e criação de novos empregos em regiões próximas. Já em economias de menor desenvolvimento, a correlação é neutra ou negativa, aumentando o risco de desemprego estrutural e migração forçada. O impacto, portanto, não é apenas setorial; ele também é geográfico e pode ampliar desigualdades entre países e entre regiões dentro de um mesmo país.

Novas ocupações estão surgindo, como Prompt Engineer e AI Operations Manager (ou Supervisor de Sistemas de IA). Essas funções exigem competências de estruturação de problemas, avaliação de qualidade e julgamento contextual, habilidades que transcendem formações tradicionais. No entanto, o acesso a essas novas oportunidades é restrito: grande parte das vagas exige background em áreas STEM, e a concentração geográfica em grandes pólos tecnológicos limita o alcance para profissionais de outras regiões. Paralelamente, ocupações tradicionais de suporte administrativo e operacional, como call centers, estão sendo reduzidas em ritmo acelerado por agentes conversacionais, gerando desemprego estrutural para profissionais com anos de experiência e identidade profissional construída nessa atividade.

A reconversão ocupacional enfrenta barreiras significativas. Estudos mostram que a taxa de sucesso na transição para novas funções é fortemente influenciada pela idade: trabalhadores mais jovens conseguem se reabsorver com mais facilidade, enquanto profissionais de meia-idade e sênior enfrentam dificuldades maiores. Programas de upskilling com melhores resultados combinam treinamento técnico com suporte psicológico, garantia de recolocação e flexibilidade para quem precisa continuar trabalhando. No entanto, o investimento necessário por trabalhador é substancial, e a maioria dos países e empresas ainda investem muito abaixo do que seria necessário para uma transição em escala.

O sistema educacional formal também apresenta um hiato importante. A velocidade de mudança do mercado de trabalho contrasta com os ciclos longos de atualização curricular nas universidades e instituições de ensino técnico. Mesmo onde há iniciativas de adaptação,

persiste uma desconexão entre as competências ensinadas e as que o mercado efetivamente demanda, especialmente competências meta-cognitivas como pensamento sistêmico, ética e compreensão do contexto humano em que a IA é aplicada.

Pesquisas recentes sobre o impacto da IA na formação de habilidades revelam um risco adicional. O uso de assistência de IA em tarefas de codificação com novas bibliotecas melhora a produtividade em alguns casos, mas pode comprometer significativamente a compreensão conceitual, a leitura de código e as habilidades de depuração. Os participantes que delegaram excessivamente as tarefas à IA apresentaram redução de até 17% nos escores de avaliação de habilidades. São identificados padrões de interação com IA que preservam a aprendizagem quando há engajamento cognitivo ativo, como perguntas conceituais e explicações solicitadas. Isso sugere que a produtividade aumentada pela IA não é um atalho para a competência e que o uso descuidado pode gerar déficits de habilidade de longo prazo, especialmente entre profissionais juniores (SHEN; TAMKIN, 2026).

O relatório Spark 2026 CIO Survey reforça que a “ansiedade de deslocamento” permanece um tema central no cenário de 2026. Organizações bem-sucedidas estão respondendo com um novo contrato social: “Guaranteed Upskilling” (reinvestimento das economias geradas pela automação no desenvolvimento humano), “Transparency Protocols” (visibilidade total dos funcionários sobre os agentes de IA sendo implantados) e o incentivo à “Creative Autonomy”, permitindo que humanos dediquem cerca de 20% do tempo a explorações não lineares que a IA não consegue simular. A resiliência organizacional em 2026 depende fortemente da confiança dos funcionários. O relatório recomenda ainda um “Systemic Human Capital Overhaul”, no qual o CHRO atua em conjunto com líderes de tecnologia para desmontar descrições de cargos da era industrial, redesenhar métricas de desempenho com foco em resultados (em vez de volume de output) e capacitar a média gerência na nova disciplina de “AI Collaboration Design”, a orquestração de equipes híbridas humano-máquina (SPARK, 2026).

Essas dinâmicas se combinam em múltiplas dimensões de desigualdade: geográfica, educacional, etária, de gênero e étnica. Um profissional que acumula desvantagens em mais de uma dessas dimensões enfrenta probabilidades muito reduzidas de acesso a oportunidades de reconversão de qualidade. O risco mais grave é a formação de uma “geração perdida”, uma

coorte de trabalhadores deslocados que, por barreiras estruturais, permanece em desemprego de longo prazo ou subemprego, com consequências duradouras sobre renda, saúde e coesão social.

2.6 Síntese

Tomadas em conjunto, as cinco dimensões analisadas neste capítulo revelam um padrão consistente na transição para sistemas baseados em IA agêntica. Embora essa transição traga ganhos concretos de produtividade e capacidade operacional, ela expõe lacunas estruturais profundas. A ausência de topologias que equilibrem autonomia técnica dos sistemas com supervisão humana efetiva, transparência e alinhamento de valores emerge como o principal estrangulamento.

Tabela 1 - Principais lacunas por dimensão na transição para sistemas de IA agêntica

Dimensão	Lacuna	Consequência
Indústria e Aplicações	Ausência de topologia arquitetural unificada e de frameworks estruturados de integração e supervisão de agentes	Risco de Agent Sprawl, paradoxo da IA generativa (alta adoção sem impacto mensurável) e alto índice de cancelamento de projetos
Desenvolvimento de Software	Avanços concentrados na geração de código, com defasagem clara em engenharia de requisitos, arquitetura e governança de orquestração multi-agente	Degradação de qualidade arquitetural (Architectural Mimicry), vulnerabilidades frequentes em código gerado por IA e perda de eficiência em sistemas complexos
Usuário e Experiência	Falta de transparência, explicabilidade e mecanismos claros de controle em interfaces generativas e hiper-adaptativas	Redução da confiança do usuário, sensação de perda de agência, fenômeno de user experience reversion e tensões com privacidade e consentimento

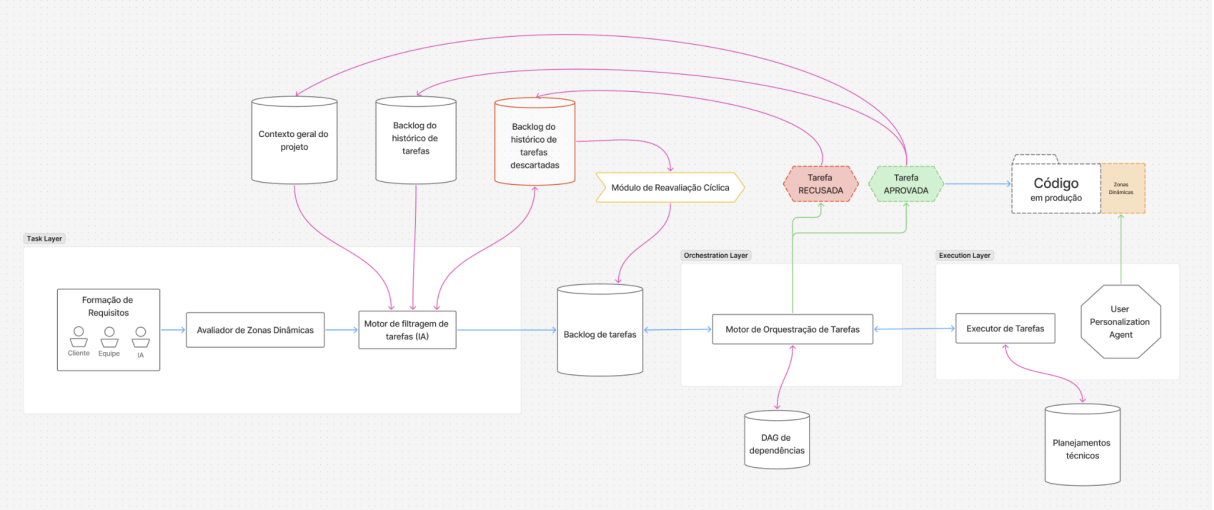
Segurança, Ética e Governança	Novos vetores de ataque (prompt injection e manipulação de intenções), amplificação de vieses e governança frágil para sistemas que executam ações autônomas	Aumento expressivo de riscos de segurança e éticos, ambiguidade na responsabilidade legal e dificuldades na aplicação de regulações
Mercado de Trabalho	Hiato temporal entre o ritmo acelerado de deslocamento de empregos e a capacidade real de reconversão da força de trabalho, agravado por desigualdades estruturais	Risco de desemprego estrutural prolongado, formação de uma geração perdida e déficits de competências cognitivas de longo prazo em profissionais juniores

Fonte: Elaboração própria do autor.

Essas lacunas não são isoladas, mas interdependentes, formando um ciclo que limita o potencial real da IA agêntica e expõe riscos significativos de adoção descontrolada. Diante desse cenário, surge a necessidade urgente de um framework híbrido que integre de forma responsável a autonomia dos agentes com mecanismos robustos de governança humana.
É exatamente essa lacuna central que a topologia PASE (Personalized Adaptive Software Ecosystem), proposta no próximo capítulo, visa preencher de maneira sistemática e prática.

3. TOPOLOGIA PASE: PERSONALIZED ADAPTIVE SOFTWARE ECOSYSTEM

3.1 Introdução e Visão Geral da Arquitetura



A análise das cinco dimensões do capítulo anterior apontou para um mesmo problema. A transição para o software agêntico já produz ganhos concretos de produtividade, mas ainda não dispõe de uma topologia arquitetural que concilie a autonomia crescente dos sistemas de IA com a supervisão, o controle e o alinhamento humano. Essa carência aparece de forma interligada nas cinco dimensões analisadas, da proliferação descontrolada de agentes à pressão sobre o trabalho humano. É para preencher essa lacuna que este capítulo propõe a PASE (Personalized Adaptive Software Ecosystem).

A PASE é uma topologia de software híbrida humano-IA, voltada a estruturar o ciclo de vida completo de uma funcionalidade, da originação da ideia à sua integração e personalização em produção, sob um regime de autonomia supervisionada. A topologia não busca a automação total nem reduz a IA a um papel apenas assistivo. Ela organiza a colaboração entre agentes autônomos e operadores humanos sob um princípio condutor: a IA propõe, decompõe e executa, enquanto o ser humano governa as decisões críticas e responde por elas.

Quatro princípios de projeto sustentam essa concepção. O primeiro é a autonomia supervisionada (human-in-the-loop), que mantém o julgamento humano nos pontos de maior incerteza e impacto, em linha com o modelo de parceria humano-IA no qual a supervisão é indispensável para decisões sensíveis (BURTE, 2025). O segundo é a contenção da generatividade: a imprevisibilidade dos modelos de linguagem é aproveitada onde agrega valor e bloqueada onde ameaça a integridade, combinando camadas determinísticas de segurança com geração probabilística, na linha da integração neuro-simbólica apontada como

direção promissora para a IA agêntica (ABOU ALI; DORNAIKA, 2025). O terceiro é a rastreabilidade integral, que preserva a proveniência de cada decisão e artefato ao longo do fluxo. O quarto é o alinhamento entre arquitetura e tarefa, segundo o qual a eficácia de um sistema multi-agente depende menos do número de agentes do que da adequação entre a estrutura de coordenação e a natureza do problema (KIM et al., 2026).

Para operacionalizar esses princípios, a PASE estrutura-se em três camadas sequenciais e complementares. A Camada de Tarefas (Task Layer) é o ponto de entrada: capta demandas de fontes heterogêneas, filtra-as quanto a redundância, coerência e valor e as qualifica em um backlog de tarefas candidatas. A Camada de Orquestração (Orchestration Layer) é o plano de controle: prioriza as tarefas, preserva a supervisão humana, modela suas interdependências em um grafo acíclico direcionado (DAG) e gerencia o ciclo de vida de cada uma até a entrega. A Camada de Execução (Execution Layer) é o nível operacional: converte as diretrizes aprovadas em código dentro de ambientes isolados e, por meio do User Personalization Agent (UPA), entrega a hiper-personalização em tempo de execução sem expor o núcleo do sistema a código arbitrário. É nesse plano que se concentra o caráter personalizado e adaptativo que dá nome à proposta.

Em conjunto, as três camadas materializam a resposta deste trabalho às suas questões de pesquisa: a PASE aposta na via híbrida, em que a autonomia técnica é ampla, mas permanece subordinada a pontos de governança humana, em vez de tratar supervisão, controle e rastreabilidade como requisitos não-funcionais difusos.

As seções seguintes detalham cada camada da topologia, descrevendo seus componentes, seus fluxos de dados e as decisões de projeto que sustentam o equilíbrio entre capacidade e confiabilidade da PASE.

3.2 Camada de Tarefas (Task Layer)

A Camada de Tarefas constitui o ponto de entrada da topologia PASE e tem por finalidade transformar demandas brutas e dispersas em um backlog estruturado, qualificado e estrategicamente alinhado ao ecossistema. É nela que se materializa um princípio central da arquitetura, que é equilibrar a abertura criativa na originação de demandas com o rigor na sua

seleção: a camada maximiza o alcance na captura de necessidades, admitindo insumos desestruturados ou de alta abstração, e em seguida submete cada proposta a um processo de filtragem progressiva, no qual redundâncias, incoerências e itens de baixo valor são descartados antes de alcançarem as camadas subsequentes.

Para tanto, a camada organiza-se em três blocos funcionais encadeados. A Formação de Requisitos origina demandas a partir de três fluxos complementares: agentes externos (usuários), agentes internos (equipe) e a própria Inteligência Artificial. Além disso, há um filtro que o Avaliador de Zonas Dinâmicas determina se cada demanda deve ser atendida por uma funcionalidade nativa estática ou pela abertura de uma área parametrizável, articulando esta camada à hiper-personalização provida em tempo de execução. O Motor de Filtragem de Tarefas atua como validador de segurança e viabilidade, normalizando os insumos heterogêneos e submetendo-os a camadas sucessivas de memória histórica, coerência sistêmica e qualificação por metadados. Por fim, o produto final é um conjunto de tarefas candidatas, anotadas por impacto, complexidade e confiança, pronto para ser priorizado e despachado pela Camada de Orquestração.

3.2.1 Formação de Requisitos

A eficácia do ecossistema depende da diversidade e da riqueza dos insumos que alimentam o Motor de Filtragem. Esta camada é projetada para capturar desde necessidades imediatas de correção até inovações disruptivas, operando através de três fluxos principais de originação: Agentes Externos (Usuários), Agentes Internos (Equipe) e a própria Inteligência Artificial.

Ao contrário do Motor de Filtragem, que prioriza o rigor e a consistência, a Etapa de Formação de Requisitos é otimizada para a maximização do alcance, permitindo que propostas desestruturadas ou de alta abstração entrem no pipeline para posterior refinamento.

3.2.1.1 Fluxo de Agentes Externos (Usuários)

Este fluxo captura necessidades reais de quem utiliza o software na ponta.

a. Captação Ativa: Interfaces simplificadas (modais) que permitem sugestão de melhorias. Para garantir a frequência e a qualidade dos insumos, a arquitetura prevê uma estratégia de engajamento onde colaborações são recompensadas, em especial as efetivamente convertidas em funcionalidades. A fim de viabilizar a escalabilidade, a qualidade inicial da proposta é avaliada por um modelo de linguagem de baixa latência, que define a recompensa prévia e filtra ruídos antes do processamento denso.
b. Captação Passiva: O sistema utiliza IA para monitorar os mais recentes tickets, interações de suporte e outros dados públicos e não sensíveis do usuário, como comentários e postagens. O objetivo é extrair intenções de uso a partir de dores relatadas, convertendo reclamações informais em propostas de tarefas estruturadas (BURTE, 2025).

3.2.1.2 Fluxo de Agentes Internos (Equipe)

Representa as inserções manuais dos agentes que possuem conhecimento do software.

a. Sugestões Funcionais: Demandas de novas funcionalidades. São de natureza propositiva e qualquer agente interno pode propor, independente do seu nível de conhecimento do software. Assim, estas sugestões seguem o fluxo integral do Motor de Filtragem de Tarefas para garantir a qualidade.
b. Sugestões de Engenharia (Bypass): Demandas de refatoração, correção de débitos técnicos ou manutenção de infraestrutura. Como são de natureza técnica deliberada, estas tarefas ignoram as camadas de filtragem do Motor de Filtragem de Tarefas e seguem diretamente da normalização para a camada final de metadados, garantindo agilidade operacional. O acesso a este fluxo deve ser restrito por uma Etapa de Autorização, garantindo apenas profissionais qualificados.

3.2.1.3 Fluxo da Inteligência Artificial

Este fluxo posiciona a IA como um agente proativo na evolução do ecossistema, capaz de antecipar necessidades sem a dependência de estímulos externos. Para otimizar a relevância das propostas, este fluxo utiliza o contexto geral do projeto, mapa de funcionalidades e histórico de descartes como guias. Além disso, para mitigar vieses e expandir o horizonte criativo, a arquitetura prevê a alternância periódica de modelos de LLM.

a. Geração Exploratória: Focada na elaboração de inovações disruptivas, esta subcamada utiliza a IA configurada com alta imprevisibilidade (maiores índices de Temperature, Top-P e Top-K). O objetivo é desafiar os limites atuais da arquitetura, propondo funcionalidades que transcendem o escopo imediato. Enquanto em sistemas convencionais a "alucinação" da IA é um erro, aqui ela é utilizada como uma ferramenta de Brainstorming Sintético.
b. Análise de Lacunas: Atua de forma analítica, identificando "vazios funcionais" através do cruzamento de módulos já implementados. Ao analisar a topologia do software, a IA identifica oportunidades de integração ou expansão que ainda não foram exploradas. Por exemplo, ao detectar domínios isolados de "Controle de Gastos" e "Agendamento", o motor pode propor a criação de um "Alerta de Pagamentos Recorrentes". Esta análise garante a continuidade evolutiva e maximiza o reuso de dados entre módulos.
c. Prospecção Competitiva (Benchmarking Automatizado): Este componente executa varreduras periódicas em fontes externas (como changelogs, documentações de concorrentes e portais de tecnologia) utilizando técnicas de Web Scraping e processamento de dados não estruturados. A IA analisa as novas funcionalidades lançadas no mercado e as confronta com o estado atual do ecossistema. O objetivo não é a replicação, mas a identificação de defasagens competitivas.

3.2.2 Middleware de Normalização

Este componente atua como um tradutor universal, convertendo dados brutos e heterogêneos (um áudio de suporte, um texto informal de cliente ou um log de erro) que vem da Etapa de Formação de Requisitos em um objeto padronizado. A normalização extrai o núcleo semântico da proposta, removendo ruídos de linguagem e estruturando o requisito em um formato que as camadas subsequentes de Similaridade e Coerência possam processar com alta precisão (ZHANG et al., 2026). Sem esta etapa, a eficácia do Banco de Dados Vetorial seria comprometida pela variação na forma de escrita das fontes.

3.2.3 Avaliador de Zonas Dinâmicas

O Avaliador de Zonas Dinâmicas é o componente de análise arquitetural responsável por determinar se uma nova demanda deve ser resolvida por meio de uma funcionalidade nativa tradicional ou pela abertura de uma Zona Dinâmica no código-fonte.

Quando um requisito (originado principalmente de agentes externos) é processado, o avaliador confronta a demanda com o histórico recente de insumos e descartes estruturados da aplicação. O componente opera sob duas linhas de viabilidade técnica:

a. Funcionalidade Global Estática: Se a dor ou oportunidade identificada possuir um padrão de solução homogêneo e aplicável à maior parte da base de usuários, o sistema gera uma tarefa convencional de backlog para o desenvolvimento de código nativo permanente.
b. Abertura de Zona Dinâmica: Se o sistema identificar que o módulo afetado é alvo de reclamações frequentes, mas com soluções altamente divergentes entre os usuários (casos isolados e de cauda longa), o componente bloqueia a criação de uma funcionalidade estática. Em vez disso, ele emite uma Tarefa de Parametrização focada em transformar aquela seção do software em uma área parametrizável. Esta zona preserva seu comportamento padrão, mas torna-se qualificada para customizações em tempo de execução (ARORA et al., 2025). A tarefa delimita rigorosamente os limites de mutabilidade do código, definindo as regras de negócio e as restrições de segurança da área afetada, mantendo o conteúdo agnóstico. Na camada de Orquestração, a equipe de desenvolvimento humano pode revisar e refinar esse escopo antes de aprovar a tarefa para a execução.

3.2.4 Motor de Filtragem de tarefas

As tarefas são originadas de fontes heterogêneas (usuários, suporte, geração autônoma e equipe interna), o que garante diversidade de insumos. Contudo, para preservar a integridade do ecossistema, todas as propostas devem transitar pelo Motor de Filtragem. Este motor atua como um validador de segurança e viabilidade, onde cada camada possui autonomia para o descarte prematuro de requisitos redundantes, incoerentes ou de baixo valor. O objetivo é garantir que apenas itens de alto valor, não duplicados e estrategicamente alinhados componham o backlog de tarefas.

Nesse processo, propostas recusadas são processadas e armazenadas em um Banco de Dados Vetorial, acompanhadas de resumos sintetizados que destacam os motivos determinantes do descarte.

Para otimizar a interpretação pelo motor de IA e garantir a escalabilidade do sistema, a arquitetura prioriza a densidade de informação: em vez de armazenar dados brutos, o sistema preserva apenas o contexto crítico da rejeição. Através de técnicas de RAG (Retrieval-Augmented Generation), o motor realiza uma recuperação semântica apenas das experiências passadas mais relevantes para o desafio atual (ABOU ALI; DORNAIKA, 2025). Isso evita a sobrecarga de contexto e permite que a IA identifique padrões de erro de forma ágil, transformando o histórico de descartes em uma base de conhecimento estratégica e leve.

3.2.3.1 Módulo de Reavaliação Cíclica

Este componente opera como um processo em segundo plano (background job) responsável pela Sincronização Evolutiva do repositório histórico. A premissa fundamental é que decisões de descarte não são permanentes, mas sim contextuais.

O módulo realiza varreduras periódicas no banco de dados de propostas recusadas e também no do contexto geral do projeto, utilizando a inteligência para correlacionar antigas premissas de rejeição com as métricas e o estado atual do ecossistema. Através de uma Análise de Deriva de Contexto, o sistema identifica se mudanças na topologia do software, na base de usuários ou nos objetivos de negócio eliminaram os impedimentos que causaram o descarte original. Caso uma oportunidade de reuso seja detectada, a proposta é reintroduzida no pipeline de filtragem com um novo status de relevância, garantindo a continuidade evolutiva e evitando a "estagnação por amnésia" da arquitetura.

3.2.3.2 Camadas de Validação

O motor de filtragem proposto opera em três camadas sucessivas: Camada de Memória Histórica, Camada de Coerência Sistêmica e Camada de Qualificação e Atribuição de Metadados.

1. Camada de Memória Histórica: Focada na Análise de Similaridade e Desduplicação, esta camada correlaciona a nova proposta com funcionalidades já implementadas, itens em desenvolvimento e propostas previamente descartadas. O objetivo é impedir a redundância e garantir a evolução incremental do ecossistema, evitando que o sistema reintroduza conceitos ou experimentos que já se mostraram ineficazes.

   Neste estágio, requisitos identificados como duplicatas são descartados sem persistência adicional na base de dados. Visto que o motivo da recusa reside em um registro já existente, a nova inserção é omitida para evitar a hipertrofia da base vetorial e o surgimento de ruído semântico. Essa estratégia de higiene de dados assegura a precisão do motor, impedindo que tarefas similares poluam o contexto e garantindo que a base de conhecimento permaneça densa e eficiente.

2. Camada de Coerência Sistêmica: Atua como o primeiro filtro de viabilidade lógica, analisando a topologia e as regras de negócio do ecossistema. Seu objetivo é identificar se a proposta é coerente com o escopo atual do software. O foco aqui é identificar se uma funcionalidade sugerida viola uma regra de negócio central e se de fato faz sentido com o contexto geral do projeto. Esta camada possui autoridade de descarte imediato caso a tarefa apresente incompatibilidade técnica ou funcional irreversível com as premissas do projeto.

   Esta camada permite ajustes pontuais para alinhar a proposta às premissas do sistema, desde que sua essência funcional seja preservada. Caso a compatibilização exija uma alteração estrutural na ideia original, o motor deve optar pelo descarte para evitar que o backlog seja preenchido por sugestões descaracterizadas. Além disso, após qualquer alteração o sistema deve revalidar a proposta contra a Camada de Memória Histórica, garantindo que o ajuste não tenha gerado uma redundância que deva ser descartada.

3. Camada de Qualificação e Atribuição de Metadados: Responsável pela categorização analítica das tarefas, utilizando o contexto da etapa anterior e o conhecimento técnico geral do ecossistema. Esta camada é responsável por estruturar a inteligência de priorização através da rotulagem do requisito, sem realizar o seu descarte.

O motor de IA analisa a proposta para mapear três variáveis fundamentais que comporão os metadados do backlog:

- Impacto Estimado: O valor potencial que a tarefa agrega ao usuário ou ao negócio.
- Complexidade Esperada: Uma estimativa de magnitude da tarefa com base na natureza da funcionalidade e no contexto técnico geral (percepção de esforço relativo) (BURTE, 2025).
- Índice de Confiança: O grau de certeza da IA quanto às predições anteriores, servindo como indicador de previsibilidade à respeito da necessidade de validação externa.

Neste estágio final do motor, o requisito é persistido em uma tabela de tarefas candidatas. Requisitos que apresentem baixa confiança ou a combinação de baixo impacto e alta complexidade, recebem um marcador de incerteza. Essa sinalização servirá para a etapa subsequente de Orquestração, alertando o gestor humano sobre a necessidade de uma análise mais criteriosa (ANTHROPIC, 2026).

3.3 Camada de Orquestração (Orchestration Layer)

A Camada de Orquestração é o plano de controle da topologia PASE, posicionando-se entre a Camada de Tarefas e a Camada de Execução. Ativada de forma reativa à validação de novas tarefas ou à ocorrência de incidentes críticos em produção, ela funciona como um painel de gerenciamento que acompanha cada demanda desde a priorização até a entrega final no código. É aqui que se concentra a supervisão humana estrutural do ecossistema: o motor produz recomendações de priorização, sequenciamento e integração, mas a decisão sobre o que de fato entra em execução permanece sob controle do gestor humano.

Para coordenar múltiplas tarefas sem conflitos, a camada modela as interdependências entre elas em um Grafo Acíclico Direcionado (DAG), isolando as tarefas independentes para processamento paralelo e regulando, por limiares configuráveis, quando aguardar a conclusão de predecessores. A esse fluxo somam-se os mecanismos de governança operacional que dão robustez ao ecossistema: o controle do ciclo de vida por estados, as políticas automáticas de reexecução, as travas de custo (budget guardrails) e o Ciclo de Auditoria e Saneamento

Recorrente. Esses mecanismos preservam a consistência arquitetural e a rastreabilidade diante da natureza probabilística dos modelos de linguagem.

3.3.1 Motor de Orquestração

O motor conduz cada tarefa por um fluxo padronizado e auditável, no qual etapas automáticas de coordenação se alternam com pontos de decisão humana.

3.3.1.1 Priorização de Tarefas

A priorização das tarefas candidatas é realizada a partir dos três metadados gerados na etapa de filtragem (Impacto Estimado, Complexidade Esperada e Índice de Confiança), complementados pelo Fator de Urgência. Este fator é uma variável dinâmica calculada pelo motor com base em eventos externos, como incidentes ativos em produção, prazos de acordos de nível de serviço (SLA), criticidade de segurança ou metas de negócio. No pipeline, a precedência de análise é dada às tarefas que combinam maior impacto, maior urgência e menor complexidade.

Essa priorização funciona como uma recomendação do motor, mantendo a tomada de decisão sob controle do gestor humano (WORLD ECONOMIC FORUM; ACCENTURE, 2026). O operador pode intervir e alterar a ordem conforme a necessidade do projeto, utilizando o marcador de incerteza (vindo da filtragem) como um alerta para identificar quais requisitos exigem uma análise mais criteriosa antes da aprovação.

3.3.1.2 Modelagem de Dependências e Despacho

Após a aprovação de uma tarefa pelo gestor, o motor de orquestração envia a tarefa e solicita à camada de Execução a criação de um Documento de Planejamento Técnico. Este documento detalha as ações necessárias, incluindo os arquivos que serão afetados, o custo estimado de tokens e o tempo de execução. Com base nessas informações, o motor estrutura a tarefa como um novo nó para o Grafo Acíclico Direcionado (DAG) de dependências. Seu papel é mapear o nível de interdependência entre todas as tarefas aprovadas que ainda não foram concluídas (JIMENEZ et al., 2024). Essas dependências são representadas pelas arestas do grafo, às quais são atribuídos pesos numéricos. O grafo isola as tarefas totalmente

independentes para que sejam executadas em paralelo de forma automatizada pela camada de Execução, eliminando conflitos de código. Caso haja dependência entre tarefas, a decisão de executar em paralelo ou aguardar a conclusão dos nós predecessores é determinada por um limite estipulado pelo administrador do sistema. Essa limitação considera o peso acumulado do caminho (a soma dos pesos das arestas interconectadas). A camada de Execução realiza as tarefas em ambientes isolados utilizando ramificações (branches) distintas do repositório. Quando uma tarefa é finalizada e está pronta para ser integrada ao ramo principal do código, o motor de Orquestração envia a ordem de integração para o motor de Execução, que avalia a existência de conflitos, emitindo alertas e aplicando as rotinas de resolução necessárias.

Após a conclusão de uma tarefa, o documento de planejamento técnico das tarefas dependentes pode exigir uma atualização. Esse processo pode ser automatizado com base em um novo limiar específico, também calculado a partir dos pesos numéricos de dependência do DAG. Além disso, para requisitos de alta complexidade, o motor pode decompor o nó principal em sub-DAGs menores, garantindo que eventuais falhas em subtarefas sejam isoladas e replanejadas sem interromper o fluxo geral do grafo (KIM et al., 2026).

3.3.1.3 Homologação e Encerramento

Ao fim da atividade, a camada de Execução devolve à Orquestração os dados de telemetria (alterações de código, cobertura de testes, consumo real de tokens e tempo de processamento), junto a um ambiente temporário para testes funcionais. O desenvolvedor revisa o resultado para tomar uma das seguintes decisões de encerramento:

a. Recusa permanente: O operador insere a justificativa do descarte, enviando a tarefa e o motivo diretamente para o banco vetorial de histórico de rejeições para evitar que o erro se repita no futuro.
b. Solicitação de ajuste: O operador aponta as modificações desejadas e a tarefa retorna para a camada de Execução para um novo ciclo de refinamento.
c. Aprovação: O código é integrado ao ambiente de produção, e as alterações atualizam a base histórica e o contexto global do projeto, que são utilizados na Task Layer.

3.3.1.4 Tratamento de Falhas

Caso a camada de Execução falhe durante o processo (erros de sintaxe, quebra de restrições ou testes reprovados), o Motor de Orquestração aplica políticas automáticas de reexecução (retry). Se o limite de tentativas for atingido sem sucesso, o motor reporta o problema ao desenvolvedor humano e suspende a tarefa (e, consequentemente, a ramificação afetada no grafo) até que uma intervenção seja realizada. Acoplado a esse sistema de reexecução, o motor possui uma trava de custo financeiro (budget guardrail). Caso o acúmulo de tokens gastos nas tentativas de correção ultrapasse o orçamento previsto inicialmente no planejamento em uma margem de tolerância (ex.: 30%), a Orquestração interrompe o ciclo automático e gera um alerta de atenção para o operador, evitando o desperdício de recursos computacionais por loops de erro (SPARK, 2026).

3.3.1.5 Ciclo de Vida

Todo esse fluxo de controle é responsável por garantir a consistência do ciclo de vida de cada tarefa. Cada tarefa transiciona estritamente pelos status de: Candidata (backlog inicial), Em Planejamento (mapeamento de arquivos pela Execução), Pronta para Despacho (dependências resolvidas no DAG), Em Execução (processamento no ambiente isolado), Suspensa (falha não solucionada), Em Homologação (revisão humana pós-retorno) e Finalizada (arquivada por sucesso ou recusa permanente).

3.3.1.6 Ciclo de Auditoria e Saneamento Recorrente

Devido aos modelos de linguagem possuírem uma natureza probabilística, ao realizarem sucessivas alterações granulares no repositório, podem provocar uma despadronização arquitetural gradativa (KRAVCHUK-KIRILYUK et al., 2025) ou introduzir vulnerabilidades de forma silenciosa (CHHABRA, 2026). Por isso, o motor de Orquestração também incorpora um Ciclo de Auditoria e Saneamento Recorrente, operando em conjunto com a camada de Execução para a manutenção proativa da base de código. O motor dispara periodicamente ou sob interferência humana uma varredura geral, instruindo a camada de Execução a analisar a totalidade do software à procura de oportunidades de refatoração e falhas de segurança. Caso inconsistências sejam detectadas, o motor de orquestração gera uma nova demanda no backlog de tarefas com um marcador de que foi criada com base no código,

mas que segue o mesmo fluxo das demais. Isso garante que o ecossistema atue na correção de desvios gerados por sua própria operação sintética.

3.4 Camada de Execução (Execution Layer)

A Camada de Execução é o nível operacional da topologia PASE: o plano em que as decisões aprovadas se convertem em código efetivamente integrado ao produto. Ela abrange dois planos complementares. O primeiro é o Executor de Tarefas, que recebe as diretrizes da Orquestração e as transforma em alterações de código dentro de ambientes isolados. O segundo é o plano de personalização em tempo de execução, conduzido pelo User Personalization Agent (UPA) sobre as Zonas Dinâmicas, responsável por entregar a hiper-personalização do ecossistema sem comprometer a integridade e a segurança do núcleo do sistema.

Esses dois planos compartilham um mesmo princípio de projeto, que é confinar a generatividade da IA a fronteiras explícitas e verificáveis, mas operam em regimes distintos: o Executor atua sobre o código-fonte permanente, sob controle da Orquestração, enquanto o UPA atua de forma autônoma sobre áreas previamente parametrizadas, em tempo de execução e isolado do fluxo de requisitos globais.

3.4.1 Executor de Tarefas

Esta camada segue o princípio do isolamento: não possui visibilidade sobre as regras de negócio globais ou sobre a priorização do backlog. Funciona como um ambiente fechado que recebe uma instrução de entrada, resolve o problema técnico por meio de ciclos locais de correção e devolve os resultados formatados para a Orquestração.

A Camada de Execução foca em contratos de entrada e saída e em restrições de ambiente, sendo agnóstica em relação à estratégia interna de geração (seja por prompt único, agentes especializados ou pipelines dinâmicos e como eles se relacionam) (ANTHROPIC, 2026).

O ciclo de trabalho desta camada é dividido em duas fases principais: a Fase de Planejamento Técnico e a Fase de Execução.

3.4.1.1 Fase de Planejamento Técnico

Antes de uma tarefa ser integrada ao Grafo Acíclico Direcionado (DAG) da Orquestração, a Camada de Execução realiza uma análise estática do repositório para avaliar o impacto do requisito enviado. O motor analisa o estado atual do código e gera um Documento de Planejamento Técnico, que é composto por:

- Mapeamento de impacto (Onde): Identificação exata de quais arquivos e módulos do sistema precisarão ser modificados, excluídos ou criados. Essa informação é enviada à camada de Orquestração, que a utiliza para identificar conflitos e montar o Grafo Acíclico Direcionado (DAG) de dependências.
- Roteiro de implementação (O que fazer): O documento (ou conjunto de documentos) que descreve a estratégia técnica e as diretrizes da alteração. Ele serve como o guia principal para a execução posterior. O formato e a estrutura deste roteiro podem variar muito dependendo da estratégia específica adotada para a geração do código.
- Custo de tokens: Uma estimativa do consumo de tokens com base na complexidade do código a ser manipulado.
- Tempo de processamento: A estimativa de tempo necessária para a execução e a rodada de testes.

Essas informações permitem que o motor de Orquestração valide a trava de custo (budget guardrail) e prepare o fluxo de despacho, enquanto o roteiro fica guardado para alimentar a próxima etapa.

3.4.1.2 Fase de Execução

Após a aprovação do gestor na Orquestração, a camada inicia a fase de escrita de código utilizando diretamente o roteiro de implementação gerado na etapa anterior como insumo auxiliar. O motor de código executa as instruções descritas no plano sem a necessidade de realizar uma nova varredura completa no repositório, otimizando o tempo e o consumo de tokens. Para garantir a segurança e a integridade do sistema, todo o processo ocorre dentro de um contêiner temporário e isolado (sandbox) (CHHABRA, 2026). O ambiente realiza o clone do repositório e o checkout na ramificação específica indicada pela Orquestração, fornecendo ao motor flexibilidade para navegar pelo projeto. Dessa forma,

embora o planejamento sirva como direcionamento principal, a arquitetura não impõe barreiras rígidas de acesso, permitindo que o motor ajuste arquivos correlacionados caso surja uma necessidade técnica imprevista durante o desenvolvimento.

Dentro desse ambiente isolado, a camada executa um ciclo de correção local, onde o código gerado passa por análises automáticas de sintaxe, qualidade e segurança antes de interagir com o ecossistema externo. Se algum problema for detectado nessas checagens estáticas, a correção é feita localmente dentro do próprio contêiner, sem onerar o fluxo da Orquestração. A validação final da tarefa baseia-se no princípio da testabilidade autônoma. O motor é responsável por executar ou atualizar a malha de testes aplicável à modificação, sendo o sucesso absoluto de execução e a garantia de que nenhuma funcionalidade antiga foi corrompida através de testes de regressão premissas obrigatórias para validar a entrega (BURTE, 2025).

3.4.1.3 Retorno de Dados e Homologação

Finalizadas as validações, a camada encerra o ambiente isolado e organiza os resultados em um pacote de dados padronizado enviado de volta ao barramento de comunicação da Orquestração. Este retorno contém o status determinístico da operação, classificado entre sucesso ou falha, acompanhado do arquivo contendo as alterações estruturadas em formato de diff, além das evidências de logs dos testes e das ferramentas de segurança. Para garantir a rastreabilidade total e permitir que o desenvolvedor assuma o controle da tarefa se necessário, o pacote inclui o link direto para a ramificação ou pull request no GitHub e o link de acesso ao histórico de interações e chat da inteligência artificial durante a execução do código. O pacote também consolida a telemetria detalhada com o consumo real de tokens e o tempo de processamento em milissegundos.

Por fim, a camada disponibiliza uma instância de visualização temporária (preview environment), permitindo que o desenvolvedor humano realize testes funcionais na interface ou na API para homologar o resultado em tempo de execução antes que a Orquestração efetue a integração final no ramo principal do código.

3.4.2 User Personalization Agent (UPA)

Uma vez que a tarefa de Abertura de Zona Dinâmica é aprovada e implementada na base de código, a topologia PASE delega o gerenciamento dessas áreas a uma camada externa de tempo de execução (runtime), controlada pelo User Personalization Agent (UPA).

O UPA opera de forma autônoma e isolada do fluxo de requisitos globais. O agente monitora o comportamento e as interações individuais de cada usuário e, ao detectar anomalias comportamentais via telemetria de interface (como cliques repetitivos por frustração, tempo excessivo para conclusão de tarefas ou taxas elevadas de erro de validação) (JAMALI et al., 2025) em módulos que possuem Zonas Dinâmicas homologadas, injeta payloads declarativos. Essa injeção também pode ser feita com base no monitoramento de mensagens desse usuário (como em tickets). Assim, o código dinâmico é criado e o componente personalizado aparece para o usuário nessa Zona Dinâmica.

Dessa forma, o ecossistema alcança a hiper-personalização (BIENIEK et al., 2024) sem a necessidade de geração de código arbitrário pela IA em áreas não preparadas, preservando a integridade e a segurança do núcleo do sistema.

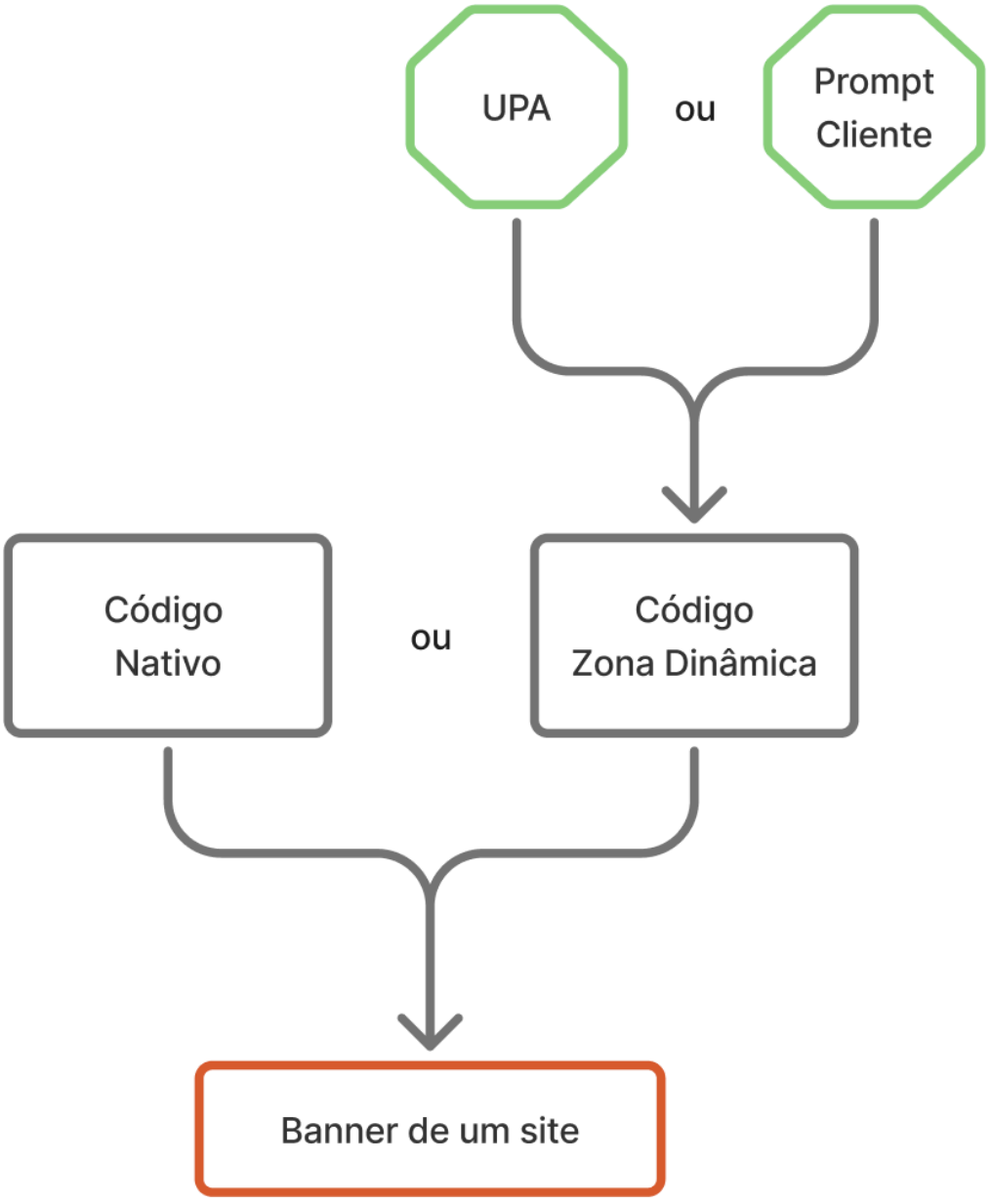
3.4.2.1 Tipologia e Governança Avançada das Zonas Dinâmicas

Para garantir a estabilidade do sistema, as Zonas Dinâmicas são classificadas em duas naturezas de acionamento:

- Zonas Dinâmicas Autônomas: Áreas controladas de forma implícita e proativa pelo UPA por meio da análise contínua de telemetria. Elas modificam fluxos e posições de componentes de interface sem a necessidade de comandos explícitos do usuário, atuando apenas quando há uma necessidade identificada.
- Zonas Dinâmicas Sob Demanda: Seguem uma abordagem baseada em intenção direta, na qual o usuário atua como co-criador da interface utilizando linguagem natural. Este fluxo gera visões personalizadas sob medida, de forma análoga aos widgets gerativos orientados por prompts de sistemas operacionais modernos (ARORA et al., 2025).

Para mitigar riscos de inputs inválidos nessas zonas, a arquitetura separa restrições abstratas (Soft Constraints) de contratos rígidos (Hard Constraints). O UPA atua no nível das

Soft Constraints, utilizando o contexto de negócio para projetar a interface de forma flexível. A validação definitiva, contudo, é executada de forma estática pelo código da Zona Dinâmica (Hard Constraints). Se o payload gerado violar as regras desse contrato, como o uso de parâmetros inválidos ou o estouro de caracteres, a Zona Dinâmica aborta imediatamente a renderização em tempo de execução.



No exemplo da imagem, considere um código nativo que exibe um banner atrelado a uma Zona Dinâmica, cujo contrato rígido aceita exclusivamente textos de até 30 caracteres e

links internos do sistema. Se o UPA ou um comando do usuário tentar submeter um payload com um texto de 500 caracteres ou um endereço externo, a Zona Dinâmica intercepta a inconformidade antes da renderização. O sistema bloqueia a exibição do conteúdo malformado e retorna a visualização padrão do código nativo, preservando a estabilidade da aplicação.

A abrangência dessas zonas ultrapassa a exibição de componentes visuais isolados, sendo capaz de orquestrar fluxos dinâmicos completos. Uma tarefa pode delimitar um processo inteiro de múltiplas etapas como ponto de extensão. Nesses cenários, o payload declarativo injetado não define apenas a interface de uma tela, mas sim uma máquina de estados contendo as regras de navegação, validações de campos e condicionais de transição entre telas. Em vez de injetar um código de programação puro (como JavaScript), o que comprometeria a segurança, utiliza-se um arquivo de configuração estruturado (como JSON). Isso garante a flexibilidade da lógica de negócio e do encadeamento de fluxos, protegendo a aplicação contra vulnerabilidades críticas, como ataques de injeção de scripts (XSS).

Para garantir a resiliência operacional, as Zonas Dinâmicas operam sob isolamento de escopo e o princípio do menor privilégio. O consumo de dados é mediado por um SDK estritamente tipado que bloqueia qualquer tentativa de escrita arbitrária ou acesso indevido ao banco de dados principal do produto. Adicionalmente, nas abordagens sob demanda, os prompts enviados pelo usuário passam por uma esteira de validação prévia que intercepta intenções maliciosas e ataques de injeção de prompt (prompt injection) antes do processamento pelo agente (CHHABRA, 2026).

3.4.2.2 Interoperabilidade Cross-Zone e Conexão Sistêmica

Para viabilizar a criação de soluções complexas, a arquitetura prevê a comunicação bidirecional entre diferentes zonas dinâmicas e o código estático por meio de um Barramento de Eventos Local (Event Bus) embutido no cliente da aplicação. Quando um componente dinâmico sofre uma alteração (como um widget de filtragem customizada), ele publica uma notificação de mudança de estado no barramento. Outras zonas dinâmicas inscritas ou elementos nativos do sistema interceptam o evento e atualizam seu comportamento de forma sincronizada. Na prática, essa estrutura permite que a interação do usuário com um filtro

gerado em uma zona dinâmica controle e atualize diretamente uma tabela nativa e estática da aplicação que foi configurada para escutar o Event Bus.

4. CONCLUSÃO

Este trabalho partiu de uma constatação e de uma tensão. A constatação é que a Inteligência Artificial deixou de ser uma ferramenta passiva e passou a atuar como um agente capaz de planejar, decidir e executar trabalho de software de forma autônoma, deslocando o ecossistema de um paradigma internet-based para um modelo agent-based (SAROYA; MACNEILL, 2026). A tensão é que essa autonomia crescente, embora promissora, esbarra na necessidade permanente de supervisão, controle e alinhamento humano. A partir disso, o estudo investigou duas questões: se o futuro seria de ecossistemas totalmente autônomos ou de sistemas híbridos humano-IA, e como projetar uma topologia de software capaz de equilibrar autonomia e supervisão.

A revisão conduzida ao longo de cinco dimensões (indústria e aplicações, desenvolvimento de software, experiência do usuário, segurança/ética/governança e mercado de trabalho) revelou um padrão consistente: em todas elas, a capacidade técnica já demonstrada contrasta com a ausência de estruturas que a tornem confiável. Disso decorre a lacuna central e interdependente que o trabalho identificou: a falta de uma topologia que equilibre a autonomia técnica dos sistemas com supervisão humana efetiva, transparência e alinhamento de valores.

Como contribuição para preencher essa lacuna, este trabalho propôs a PASE (Personalized Adaptive Software Ecosystem), uma topologia híbrida humano-IA organizada em três camadas. A Camada de Tarefas disciplina a originação e a filtragem de demandas, convertendo insumos heterogêneos em um backlog qualificado e contendo a proliferação descontrolada de iniciativas. A Camada de Orquestração concentra a supervisão humana e a governança operacional, pois transforma a priorização em recomendação sujeita a decisão

humana e coordena a execução por meio de um grafo de dependências, travas de custo e um ciclo de auditoria que combate a degradação arquitetural silenciosa. A Camada de Execução isola a geração de código em ambientes controlados e confina a hiper-personalização a zonas dinâmicas verificáveis, entregando adaptação ao usuário sem abrir mão da integridade e da segurança do núcleo.

Essa construção permite responder às questões que motivaram a pesquisa. Quanto à primeira, a evidência analisada e o desenho proposto convergem para a via híbrida. A substituição integral do humano não se confirma na prática, já que engenheiros que adotam IA relatam, em geral, ampliação, e não redução, de responsabilidades (ANTHROPIC, 2026), e a eficácia dos sistemas multi-agente mostra-se condicionada ao alinhamento entre a arquitetura de coordenação e a estrutura da tarefa (KIM et al., 2026). O cenário mais provável, e mais desejável, é o de ecossistemas em que a autonomia é ampla e, ao mesmo tempo, subordinada a pontos de governança humana. Quanto à segunda, a PASE responde com um desenho concreto, no qual supervisão, segurança e rastreabilidade não são tratadas como requisitos não-funcionais difusos, mas incorporadas como mecanismos estruturais distribuídos pelas três camadas.

Os resultados devem ser lidos à luz de suas limitações. A PASE é, neste estágio, uma proposta conceitual, que não foi submetida a validação empírica e cuja eficácia em escala real permanece uma hipótese a ser testada. A literatura adverte que sistemas multi-agente podem degradar-se em tarefas que exigem raciocínio profundo e coordenação intensa (HE; TREUDE; LO, 2024) e que a coordenação gera retornos decrescentes quando aplicada ao problema errado (KIM et al., 2026), riscos que um protótipo precisaria enfrentar. Além disso, o custo de inferência, a maturidade dos dados organizacionais e a curva de adaptação das equipes podem condicionar fortemente a viabilidade prática da topologia.

Como desdobramentos, abrem-se três frentes principais. A primeira é a implementação de um protótipo funcional de cada camada. A segunda é a definição de métricas que permitam avaliar empiricamente o equilíbrio entre autonomia e supervisão, como a taxa de intervenção humana, a densidade de retrabalho e a incidência de regressões. A terceira é a investigação dos efeitos sociotécnicos da topologia sobre as equipes que a operam, em especial sobre a formação de competências, cuja preservação depende de engajamento

cognitivo ativo e não da simples delegação à IA (SHEN; TAMKIN, 2026). Com isso, a PASE pretende oferecer um vocabulário e uma estrutura para que a confiabilidade seja tratada como requisito de projeto desde o início, e não como ajuste posterior.

REFERÊNCIAS

ABOU ALI, Mohamad; DORNAIKA, Fadi. **Agentic AI: a comprehensive survey of architectures, applications, and future directions.** arXiv, 2025. Disponível em: https://arxiv.org/abs/2510.25445. Acesso em: 12 jan. 2026.

ANTHROPIC. **2026 Agentic Coding Trends Report: how coding agents are reshaping software development.** San Francisco: Anthropic, 2026. Disponível em: https://resources.anthropic.com/hubfs/2026%20Agentic%20Coding%20Trends%20Report.pdf. Acesso em: 05 fev. 2026.

ARORA, Piyush et al. **Self-Evolving Systems: moving beyond deterministic interfaces to adaptive generative interfaces.** Google Research, 2025. Disponível em: https://storage.googleapis.com/gweb-research2023-media/pubtools/1034946.pdf. Acesso em: 18 nov. 2025.

BIENIEK, J.; RAHOUTI, M.; VERMA, D. C. **Generative AI in Multimodal User Interfaces: trends, challenges, and cross-platform adaptability.** arXiv, 2024. Disponível em: https://arxiv.org/abs/2411.10234. Acesso em: 30 abr. 2026.

BURTE, Sandeep. **AI-Powered Software Development Life Cycle: from requirements to maintenance.** AI Systems Engineering, Bengaluru, 2025. Disponível em: https://aise.cultechpub.com/index.php/aise. Acesso em: 30 abr. 2026.

CHHABRA, Anshuman et al. **Agentic AI Security: threats, defenses, evaluation, and open challenges.** IEEE Access, v. 14, 2026. DOI: 10.1109/ACCESS.2026.3675554. Disponível em: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=11447227. Acesso em: 20 abr. 2026.

CROWDSTRIKE. **CrowdStrike 2026 Global Threat Report.** Austin, TX: CrowdStrike Holdings, 2026. Disponível em: https://www.crowdstrike.com/en-us/global-threat-report/. Acesso em: 18 abr. 2026.

GARTNER. **Gartner predicts 40% of enterprise apps will feature task-specific AI agents by 2026, up from less than 5% in 2025.** Stamford: Gartner, 2025a. Disponível em: https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025. Acesso em: 13 abr. 2026.

GARTNER. **Gartner predicts over 40% of agentic AI projects will be canceled by end of 2027.** Stamford: Gartner, 2025b. Disponível em: https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027. Acesso em: 13 abr. 2026.

GARTNER. **Gartner forecasts supply chain management software with agentic AI will grow to $53 billion in spend by 2030.** Stamford: Gartner, 2026. Disponível em: https://www.gartner.com/en/newsroom/press-releases/2026-04-07-gartner-forecasts-supply-chain-management-software-with-agentic-ai-will-grow-to-53-billion-in-spend-by-2030. Acesso em: 13 abr. 2026.

GOOGLE CLOUD. **State of AI-assisted Software Development 2025.** Mountain View: Google Cloud, 2025. (DORA Report, v. 2025.2). Disponível em: https://services.google.com/fh/files/misc/2025_state_of_ai_assisted_software_development.pdf. Acesso em: 13 abr. 2026.

HE, Junda et al. **LLM-Based Multi-Agent Systems for Software Engineering: literature review, vision and the road ahead.** arXiv, 2024. Disponível em: https://arxiv.org/abs/2404.04834. Acesso em: 19 out. 2025.

JAMALI, H. et al. **AI-powered adaptive learning interfaces: a user experience study in education platforms.** Frontiers in Computer Science, Lausanne, v. 7, p. 1672081, 2025. DOI: 10.3389/fcomp.2025.1672081. Disponível em: https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1672081/full. Acesso em: 10 mar. 2026.

JIMENEZ, **Carlos E. et al. SWE-bench: can language models resolve real-world GitHub issues?** In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 12., 2024, Vienna. Proceedings [...]. Vienna: OpenReview, 2024. Disponível em: https://arxiv.org/abs/2310.06770. Acesso em: 05 nov. 2025.

KIM, Yubin et al. **Towards a Science of Scaling Agent Systems.** arXiv, 2026. Disponível em: https://arxiv.org/abs/2512.08296. Acesso em: 18 jan. 2026.

KRAVCHUK-KIRILYUK, Anastasiya; GRACIOLLI, Fernanda; AMIN, Nada. **The Modular Imperative: rethinking LLMs for maintainable software.** In: ACM SIGPLAN INTERNATIONAL WORKSHOP ON LANGUAGE MODELS AND PROGRAMMING LANGUAGES (LMPL '25), 1., 2025, Singapore. Anais [...]. New York: ACM, 2025. p. 1-6. DOI: 10.1145/3759425.3763392. Disponível em: https://namin.seas.harvard.edu/pubs/lmpl-modularity.pdf. Acesso em: 13 fev. 2026.

MCKINSEY & COMPANY. **Seizing the agentic AI advantage: a CEO playbook to solve the gen AI paradox and unlock scalable impact with AI agents.** New York: McKinsey & Company, 2025a. Disponível em: https://www.mckinsey.com/capabilities/quantumblack/our-insights/seizing-the-agentic-ai-advantage. Acesso em: 22 fev. 2026.

MCKINSEY & COMPANY. **The state of AI in 2025: agents, innovation, and transformation.** New York: McKinsey & Company, 2025b. Disponível em: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai. Acesso em: 25 jan. 2026.

MUKTI, Anggit Jaya; TRISILIA, Meilinda. **AI-Powered Adaptive Interface: enhancing user experience through real-time personalization in digital platforms.** Procedia Computer Science, v. 269, p. 571-580, 2025. DOI: 10.1016/j.procs.2025.08.309. Disponível em: https://www.sciencedirect.com/science/article/pii/S1877050925026547. Acesso em: 21 jan. 2026.

SAROYA, Monti; MACNEILL, Ashley. **Vista's 2026 outlook: agentic AI and the future of enterprise software.** Austin: Vista Equity Partners, 2026. 16 p. Disponível em: https://www.vistaequitypartners.com/wp-content/uploads/2026/03/Vista-Equity-Partners_AI-Outlook-PDF-Mar-2026.pdf. Acesso em: 02 abr. 2026.

SHEN, Judy Hanwen; TAMKIN, Alex. **How AI impacts skill formation.** arXiv, 2026. Disponível em: https://arxiv.org/abs/2601.20245. Acesso em: 13 abr. 2026.

SPARK. **The Enterprise AI Transformation Journey: embracing the agentic pivot & beyond.** 2026 CIO Survey Report. Singapore: WAH Spark, 2026. Disponível em: https://wahspark.com/wp-content/uploads/2026/03/SPARK-The-Enterprise-AI-Transformation-Journey.pdf. Acesso em: 19 abr. 2026.

WORLD ECONOMIC FORUM; ACCENTURE. **Organizational Transformation in the Age of AI: how organizations maximize AI's potential.** Geneva: World Economic Forum, 2026. 43 p. (Industries in the Intelligent Age). Disponível em: https://reports.weforum.org/docs/WEF_Organizational_Transformation_in_the_Age_of_AI_How_Organizations_Maximize_AI%27s_Potential_2026.pdf. Acesso em: 04 abr. 2026.

ZHANG, Quanjun et al. **A survey on large language models for software engineering.** Science China Information Sciences, v. 69, art. 141102, 2026. DOI: 10.1007/s11432-025-4670-0. Disponível em: https://link.springer.com/article/10.1007/s11432-025-4670-0. Acesso em: 30 mar. 2026.

APÊNDICE A

Inserir apêndice (se houver).

ANEXO A

Inserir anexo se tiver